This algorithm converges rapidly (for an analysis and proof see e.g., Boyd and Vandenberghe (2004)). However, for the purpose of deep learning this is not quite so feasible, since each step of the line search would require us to evaluate the objective function on the entire dataset. This is way too costly to accomplish.

12.3.4 Summary

- Learning rates matter. Too large and we diverge, too small and we do not make progress.
- Gradient descent can get stuck in local minima.
- In high dimensions adjusting the learning rate is complicated.
- Preconditioning can help with scale adjustment.
- Newton's method is a lot faster once it has started working properly in convex problems.
- Beware of using Newton's method without any adjustments for nonconvex problems.

12.3.5 Exercises

1. Experiment with different learning rates and objective functions for gradient descent.
2. Implement line search to minimize a convex function in the interval $[a, b]$.
    1. Do you need derivatives for binary search, i.e., to decide whether to pick $[a, (a + b)/2]$ or $[(a + b)/2, b]$.
    2. How rapid is the rate of convergence for the algorithm?
    3. Implement the algorithm and apply it to minimizing $\log(\exp(x) + \exp(-2x - 3))$.
3. Design an objective function defined on $\mathbb{R}^2$ where gradient descent is exceedingly slow. Hint: scale different coordinates differently.
4. Implement the lightweight version of Newton's method using preconditioning:
    1. Use diagonal Hessian as preconditioner.
    2. Use the absolute values of that rather than the actual (possibly signed) values.
    3. Apply this to the problem above.
5. Apply the algorithm above to a number of objective functions (convex or not). What happens if you rotate coordinates by 45 degrees?

168



Discussions[168].

12.4 Stochastic Gradient Descent

In earlier chapters we kept using stochastic gradient descent in our training procedure, however, without explaining why it works. To shed some light on it, we just described the basic principles of gradient descent in Section 12.3. In this section, we go on to discuss *stochastic gradient descent* in greater detail.

```
%matplotlib inline
import math
import torch
from d2l import torch as d2l
```

12.4.1 Stochastic Gradient Updates

In deep learning, the objective function is usually the average of the loss functions for each example in the training dataset. Given a training dataset of n examples, we assume that $f_i(\mathbf{x})$ is the loss function with respect to the training example of index i, where $\mathbf{x}$ is the parameter vector. Then we arrive at the objective function

$$f(\mathbf{x}) = \frac{1}{n} \sum_{i=1}^{n} f_i(\mathbf{x}). \tag{12.4.1}$$

The gradient of the objective function at $\mathbf{x}$ is computed as

$$\nabla f(\mathbf{x}) = \frac{1}{n} \sum_{i=1}^{n} \nabla f_i(\mathbf{x}). \tag{12.4.2}$$

If gradient descent is used, the computational cost for each independent variable iteration is $O(n)$, which grows linearly with n. Therefore, when the training dataset is larger, the cost of gradient descent for each iteration will be higher.

Stochastic gradient descent (SGD) reduces computational cost at each iteration. At each iteration of stochastic gradient descent, we uniformly sample an index $i \in \{1, \ldots, n\}$ for data examples at random, and compute the gradient $\nabla f_i(\mathbf{x})$ to update $\mathbf{x}$:

$$\mathbf{x} \leftarrow \mathbf{x} - \eta \nabla f_i(\mathbf{x}), \tag{12.4.3}$$

where η is the learning rate. We can see that the computational cost for each iteration drops from $O(n)$ of the gradient descent to the constant $O(1)$. Moreover, we want to emphasize that the stochastic gradient $\nabla f_i(\mathbf{x})$ is an unbiased estimate of the full gradient $\nabla f(\mathbf{x})$ because

$$\mathbb{E}_i \nabla f_i(\mathbf{x}) = \frac{1}{n} \sum_{i=1}^{n} \nabla f_i(\mathbf{x}) = \nabla f(\mathbf{x}). \tag{12.4.4}$$

This means that, on average, the stochastic gradient is a good estimate of the gradient.

Now, we will compare it with gradient descent by adding random noise with a mean of 0 and a variance of 1 to the gradient to simulate a stochastic gradient descent.

```
def f(x1, x2):  # Objective function
    return x1 ** 2 + 2 * x2 ** 2

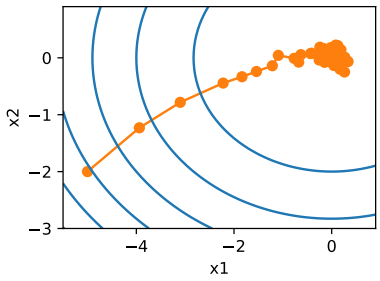
def f_grad(x1, x2):  # Gradient of the objective function
    return 2 * x1, 4 * x2
```

```
def sgd(x1, x2, s1, s2, f_grad):
    g1, g2 = f_grad(x1, x2)
    # Simulate noisy gradient
    g1 += torch.normal(0.0, 1, (1,)).item()
    g2 += torch.normal(0.0, 1, (1,)).item()
    eta_t = eta * lr()
    return (x1 - eta_t * g1, x2 - eta_t * g2, 0, 0)
```

```
def constant_lr():
    return 1

eta = 0.1
lr = constant_lr  # Constant learning rate
d2l.show_trace_2d(f, d2l.train_2d(sgd, steps=50, f_grad=f_grad))
```

```
epoch 50, x1: 0.225517, x2: -0.076646
```



As we can see, the trajectory of the variables in the stochastic gradient descent is much more noisy than the one we observed in gradient descent in Section 12.3. This is due to the stochastic nature of the gradient. That is, even when we arrive near the minimum, we are still subject to the uncertainty injected by the instantaneous gradient via $\eta \nabla f_i(\mathbf{x})$. Even after 50 steps the quality is still not so good. Even worse, it will not improve after additional steps (we encourage you to experiment with a larger number of steps to confirm this). This leaves us with the only alternative: change the learning rate η. However, if we pick this too small, we will not make any meaningful progress initially. On the other hand, if we pick it too large, we will not get a good solution, as seen above. The only way to resolve these conflicting goals is to reduce the learning rate *dynamically* as optimization progresses.

This is also the reason for adding a learning rate function `lr` into the `sgd` step function. In

the example above any functionality for learning rate scheduling lies dormant as we set the associated `lr` function to be constant.

12.4.2 Dynamic Learning Rate

Replacing η with a time-dependent learning rate $\eta(t)$ adds to the complexity of controlling convergence of an optimization algorithm. In particular, we need to figure out how rapidly η should decay. If it is too quick, we will stop optimizing prematurely. If we decrease it too slowly, we waste too much time on optimization. The following are a few basic strategies that are used in adjusting η over time (we will discuss more advanced strategies later):

$$\begin{aligned} \eta(t) &= \eta_i \text{ if } t_i \leq t \leq t_{i+1} && \text{piecewise constant} \\ \eta(t) &= \eta_0 \cdot e^{-\lambda t} && \text{exponential decay} \\ \eta(t) &= \eta_0 \cdot (\beta t + 1)^{-\alpha} && \text{polynomial decay} \end{aligned} \tag{12.4.5}$$

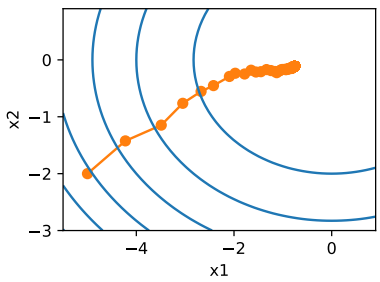
In the first *piecewise constant* scenario we decrease the learning rate, e.g., whenever progress in optimization stalls. This is a common strategy for training deep networks. Alternatively we could decrease it much more aggressively by an *exponential decay*. Unfortunately this often leads to premature stopping before the algorithm has converged. A popular choice is *polynomial decay* with $\alpha = 0.5$. In the case of convex optimization there are a number of proofs that show that this rate is well behaved.

Let's see what the exponential decay looks like in practice.

```
def exponential_lr():
    # Global variable that is defined outside this function and updated inside
    global t
    t += 1
    return math.exp(-0.1 * t)

t = 1
lr = exponential_lr
d2l.show_trace_2d(f, d2l.train_2d(sgd, steps=1000, f_grad=f_grad))
```

```
epoch 1000, x1: -0.758829, x2: -0.115584
```



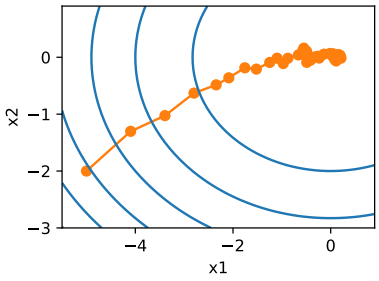
As expected, the variance in the parameters is significantly reduced. However, this comes

at the expense of failing to converge to the optimal solution $\mathbf{x} = (0, 0)$. Even after 1000 iteration steps are we are still very far away from the optimal solution. Indeed, the algorithm fails to converge at all. On the other hand, if we use a polynomial decay where the learning rate decays with the inverse square root of the number of steps, convergence gets better after only 50 steps.

```
def polynomial_lr():
    # Global variable that is defined outside this function and updated inside
    global t
    t += 1
    return (1 + 0.1 * t) ** (-0.5)

t = 1
lr = polynomial_lr
d2l.show_trace_2d(f, d2l.train_2d(sgd, steps=50, f_grad=f_grad))
```

```
epoch 50, x1: 0.144834, x2: 0.041688
```



There exist many more choices for how to set the learning rate. For instance, we could start with a small rate, then rapidly ramp up and then decrease it again, albeit more slowly. We could even alternate between smaller and larger learning rates. There exists a large variety of such schedules. For now let's focus on learning rate schedules for which a comprehensive theoretical analysis is possible, i.e., on learning rates in a convex setting. For general nonconvex problems it is very difficult to obtain meaningful convergence guarantees, since in general minimizing nonlinear nonconvex problems is NP hard. For a survey see e.g., the excellent lecture notes[169] of Tibshirani 2015.

169



12.4.3 Convergence Analysis for Convex Objectives

The following convergence analysis of stochastic gradient descent for convex objective functions is optional and primarily serves to convey more intuition about the problem. We limit ourselves to one of the simplest proofs (Nesterov and Vial, 2000). Significantly more advanced proof techniques exist, e.g., whenever the objective function is particularly well behaved.

Suppose that the objective function $f(\boldsymbol{\xi}, \mathbf{x})$ is convex in $\mathbf{x}$ for all $\boldsymbol{\xi}$. More concretely, we

consider the stochastic gradient descent update:

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \eta_t \partial_{\mathbf{x}} f(\boldsymbol{\xi}_t, \mathbf{x}), \tag{12.4.6}$$

where $f(\boldsymbol{\xi}_t, \mathbf{x})$ is the objective function with respect to the training example $\boldsymbol{\xi}_t$ drawn from some distribution at step t and $\mathbf{x}$ is the model parameter. Denote by

$$R(\mathbf{x}) = E_{\boldsymbol{\xi}}[f(\boldsymbol{\xi}, \mathbf{x})] \tag{12.4.7}$$

the expected risk and by R^* its minimum with regard to $\mathbf{x}$. Last let $\mathbf{x}^*$ be the minimizer (we assume that it exists within the domain where $\mathbf{x}$ is defined). In this case we can track the distance between the current parameter $\mathbf{x}_t$ at time t and the risk minimizer $\mathbf{x}^*$ and see whether it improves over time:

$$\begin{aligned} &\|\mathbf{x}_{t+1} - \mathbf{x}^*\|^2 \\ =&\|\mathbf{x}_t - \eta_t \partial_{\mathbf{x}} f(\boldsymbol{\xi}_t, \mathbf{x}) - \mathbf{x}^*\|^2 \\ =&\|\mathbf{x}_t - \mathbf{x}^*\|^2 + \eta_t^2 \|\partial_{\mathbf{x}} f(\boldsymbol{\xi}_t, \mathbf{x})\|^2 - 2\eta_t \left\langle \mathbf{x}_t - \mathbf{x}^*, \partial_{\mathbf{x}} f(\boldsymbol{\xi}_t, \mathbf{x}) \right\rangle. \end{aligned} \tag{12.4.8}$$

We assume that the ℓ_2 norm of stochastic gradient $\partial_{\mathbf{x}} f(\boldsymbol{\xi}_t, \mathbf{x})$ is bounded by some constant L, hence we have that

$$\eta_t^2 \|\partial_{\mathbf{x}} f(\boldsymbol{\xi}_t, \mathbf{x})\|^2 \leq \eta_t^2 L^2. \tag{12.4.9}$$

We are mostly interested in how the distance between $\mathbf{x}_t$ and $\mathbf{x}^*$ changes *in expectation*. In fact, for any specific sequence of steps the distance might well increase, depending on whichever $\boldsymbol{\xi}_t$ we encounter. Hence we need to bound the dot product. Since for any convex function f it holds that $f(\mathbf{y}) \geq f(\mathbf{x}) + \langle f'(\mathbf{x}), \mathbf{y} - \mathbf{x} \rangle$ for all $\mathbf{x}$ and $\mathbf{y}$, by convexity we have

$$f(\boldsymbol{\xi}_t, \mathbf{x}^*) \geq f(\boldsymbol{\xi}_t, \mathbf{x}_t) + \left\langle \mathbf{x}^* - \mathbf{x}_t, \partial_{\mathbf{x}} f(\boldsymbol{\xi}_t, \mathbf{x}_t) \right\rangle. \tag{12.4.10}$$

Plugging both inequalities (12.4.9) and (12.4.10) into (12.4.8) we obtain a bound on the distance between parameters at time $t + 1$ as follows:

$$\|\mathbf{x}_t - \mathbf{x}^*\|^2 - \|\mathbf{x}_{t+1} - \mathbf{x}^*\|^2 \geq 2\eta_t (f(\boldsymbol{\xi}_t, \mathbf{x}_t) - f(\boldsymbol{\xi}_t, \mathbf{x}^*)) - \eta_t^2 L^2. \tag{12.4.11}$$

This means that we make progress as long as the difference between current loss and the optimal loss outweighs $\eta_t L^2/2$. Since this difference is bound to converge to zero it follows that the learning rate η_t also needs to *vanish*.

Next we take expectations over (12.4.11). This yields

$$E\left[\|\mathbf{x}_t - \mathbf{x}^*\|^2\right] - E\left[\|\mathbf{x}_{t+1} - \mathbf{x}^*\|^2\right] \geq 2\eta_t [E[R(\mathbf{x}_t)] - R^*] - \eta_t^2 L^2. \tag{12.4.12}$$

The last step involves summing over the inequalities for $t \in \{1, \ldots, T\}$. Since the sum telescopes and by dropping the lower term we obtain

$$\|\mathbf{x}_1 - \mathbf{x}^*\|^2 \geq 2 \left(\sum_{t=1}^{T} \eta_t \right) [E[R(\mathbf{x}_t)] - R^*] - L^2 \sum_{t=1}^{T} \eta_t^2. \tag{12.4.13}$$

Note that we exploited that $\mathbf{x}_1$ is given and thus the expectation can be dropped. Last define

$$\bar{\mathbf{x}} \stackrel{\text{def}}{=} \frac{\sum_{t=1}^{T} \eta_t \mathbf{x}_t}{\sum_{t=1}^{T} \eta_t}. \tag{12.4.14}$$

Since

$$E\left(\frac{\sum_{t=1}^{T} \eta_t R(\mathbf{x}_t)}{\sum_{t=1}^{T} \eta_t}\right) = \frac{\sum_{t=1}^{T} \eta_t E[R(\mathbf{x}_t)]}{\sum_{t=1}^{T} \eta_t} = E[R(\mathbf{x}_t)], \tag{12.4.15}$$

by Jensen's inequality (setting $i = t$, $\alpha_i = \eta_t / \sum_{t=1}^{T} \eta_t$ in (12.2.3)) and convexity of R it follows that $E[R(\mathbf{x}_t)] \geq E[R(\bar{\mathbf{x}})]$, thus

$$\sum_{t=1}^{T} \eta_t E[R(\mathbf{x}_t)] \geq \sum_{t=1}^{T} \eta_t E\left[R(\bar{\mathbf{x}})\right]. \tag{12.4.16}$$

Plugging this into the inequality (12.4.13) yields the bound

$$[E[\bar{\mathbf{x}}]] - R^* \leq \frac{r^2 + L^2 \sum_{t=1}^{T} \eta_t^2}{2 \sum_{t=1}^{T} \eta_t}, \tag{12.4.17}$$

where $r^2 \stackrel{\text{def}}{=} \|\mathbf{x}_1 - \mathbf{x}^*\|^2$ is a bound on the distance between the initial choice of parameters and the final outcome. In short, the speed of convergence depends on how the norm of stochastic gradient is bounded (L) and how far away from optimality the initial parameter value is (r). Note that the bound is in terms of $\bar{\mathbf{x}}$ rather than $\mathbf{x}_T$. This is the case since $\bar{\mathbf{x}}$ is a smoothed version of the optimization path. Whenever r, L, and T are known we can pick the learning rate $\eta = r/(L\sqrt{T})$. This yields as upper bound $rL/\sqrt{T}$. That is, we converge with rate $O(1/\sqrt{T})$ to the optimal solution.

12.4.4 Stochastic Gradients and Finite Samples

So far we have played a bit fast and loose when it comes to talking about stochastic gradient descent. We posited that we draw instances x_i, typically with labels y_i from some distribution $p(x, y)$ and that we use this to update the model parameters in some manner. In particular, for a finite sample size we simply argued that the discrete distribution $p(x, y) = \frac{1}{n} \sum_{i=1}^{n} \delta_{x_i}(x) \delta_{y_i}(y)$ for some functions δ_{x_i} and δ_{y_i} allows us to perform stochastic gradient descent over it.

However, this is not really what we did. In the toy examples in the current section we simply added noise to an otherwise non-stochastic gradient, i.e., we pretended to have pairs (x_i, y_i). It turns out that this is justified here (see the exercises for a detailed discussion). More troubling is that in all previous discussions we clearly did not do this. Instead we iterated over all instances *exactly once*. To see why this is preferable consider the converse, namely that we are sampling n observations from the discrete distribution *with replacement*. The probability of choosing an element i at random is $1/n$. Thus to choose it *at least* once is

$$P(\text{choose } i) = 1 - P(\text{omit } i) = 1 - (1 - 1/n)^n \approx 1 - e^{-1} \approx 0.63. \tag{12.4.18}$$